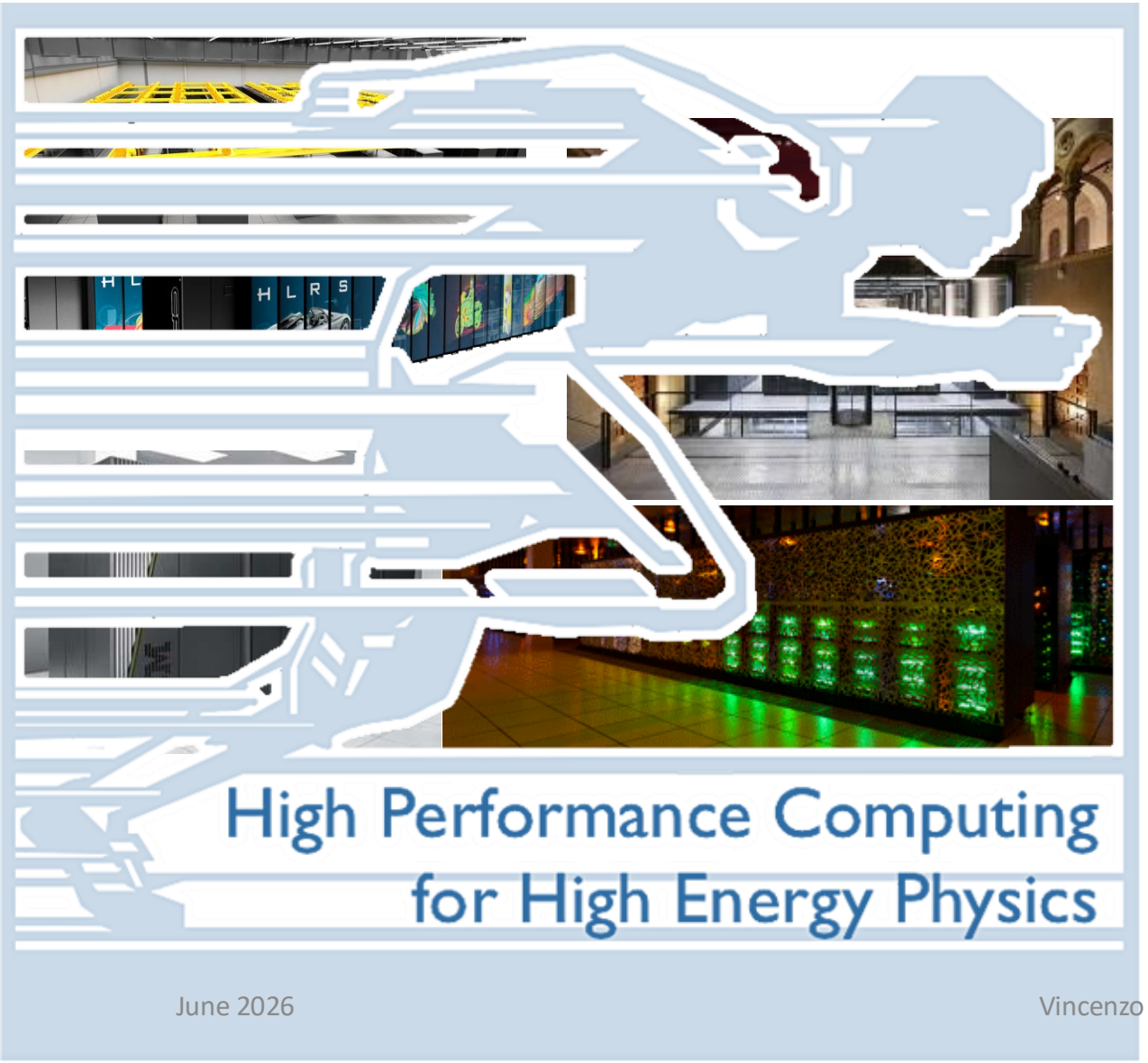
FloatingPoint Arithmetic for Scientists: Precision, Accuracy, Efficiency



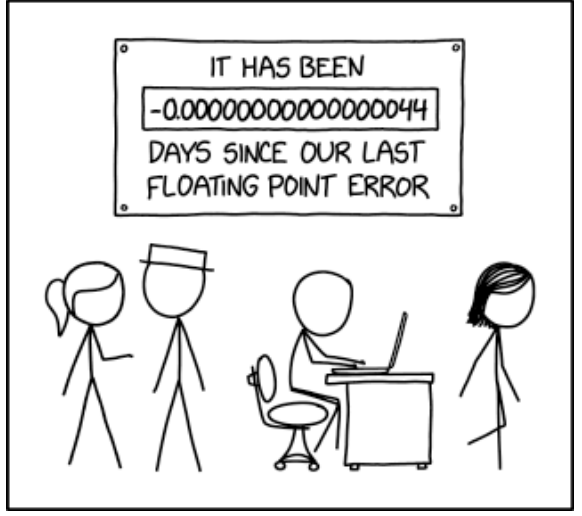
Vincenzo Innocente

CERN, June 2026

Definitions

What is a Floating Point?

https://en.wikipedia.org/wiki/Floating-point_arithmetic



- Science is based on Measurements
- The result of a Measurement is a Rational Number
  - $M_H = 125.04 \pm 0.12$ GeV = $2.2290 \cdot 10^{-25} \pm 2.1 \cdot 10^{-28}$ kg
- "Floating Point" is a way to represent Rational Numbers by a fixed number of digits (precision) multiplied by a "base" with an exponent
  - A form of "Scientific Notation"

$31 \cdot 10^{-1}$ is a FP in base 10 with "precision" 2 (# significant digits) printed in decimal

FP are usually "normalized" : the first digit is always not zero

$1.921fb6 \cdot 2^{+1}$ is a FP in base 2 with precision 24 printed in exadecimal

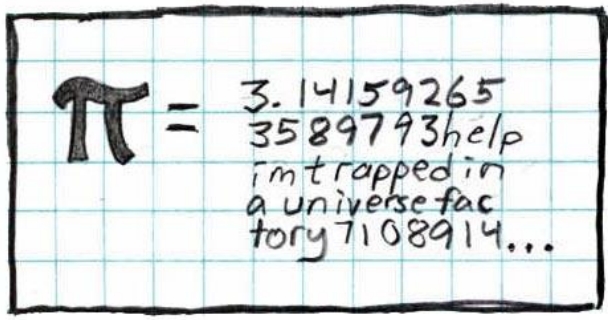
(printed in binary is a 1 followed by a string of 23bits, $1.10010010000111111011011$ in hex last digit is therefore always even)

Some Trivial properties of FP

- Each FP has a previous and a next
  - Previous of $31 \cdot 10^{-1}$ is $30 \cdot 10^{-1}$, next is $32 \cdot 10^{-1}$
  - Previous of $1.921fb6 \cdot 2^{+1}$ is $1.921fb4 \cdot 2^{+1}$, next is $1.921fb8 \cdot 2^{+1}$
  - The difference between two consecutive FP is called ULP (unit in last place)
- For each value of the exponent there is a finite number of FP
  - $10 \cdot 10^{e}$ - $99 \cdot 10^{e}$
  - $1.000000 \cdot 2^{e}$ - $1.ffffff \cdot 2^{e}$

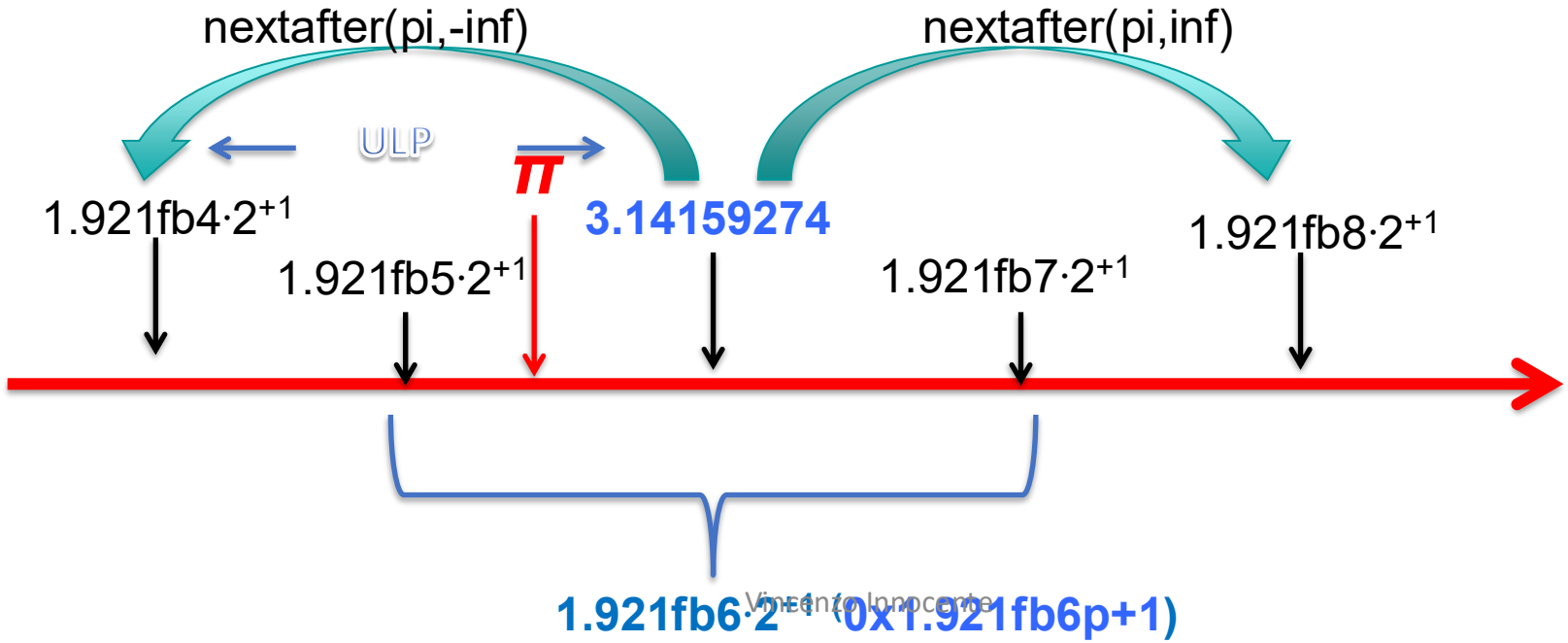
  (called “binade”)

Precision and Rounding

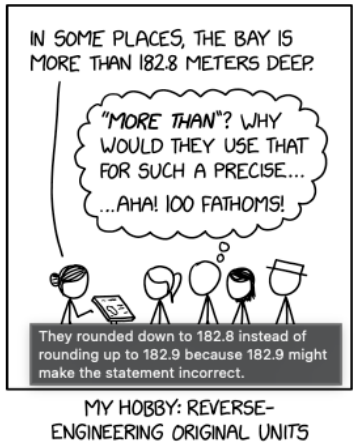


The result of a measurement is 3.1415 and its accuracy, say, 0.1.
This result needs to be rounded. Default is to round to “nearest”
3.1415 -> 3.1 (3.1515 -> 3.2)

$31 \cdot 10^{-1}$ represents all numbers in the interval $[30.5 \cdot 10^{-1}, 31.5 \cdot 10^{-1}]$
$1.921fb6 \cdot 2^{+1}$ represents all numbers in the interval $[1.921fb5 \cdot 2^{+1}, 1.921fb7 \cdot 2^{+1}]$



Rounding Algorithms



- The standard defines five rounding algorithms.
  - The first two round to a nearest value; the others are called directed roundings:
- Roundings to nearest
  - **Round to nearest, ties to even – rounds to the nearest value;** if the number falls midway it is rounded to the nearest value with an even (zero) least significant bit, which occurs 50% of the time; this is the default algorithm for binary floating-point and the recommended default for decimal
  - Round to nearest, ties away from zero – rounds to the nearest value; if the number falls midway it is rounded to the nearest value above (for positive numbers) or below (for negative numbers)
- Directed roundings
  - Round toward 0 – directed rounding towards zero (also known as truncation).
  - Round toward +∞ – directed rounding towards positive infinity (also known as rounding up or ceiling).
  - Round toward −∞ – directed rounding towards negative infinity (also known as rounding down or floor).
- A ”function” is **correctly rounded** if the result corresponds to the rounded true value
- A “function” is **faithfully rounded** if the result corresponds to one of the two FP closest to the true value

FP on a Computer

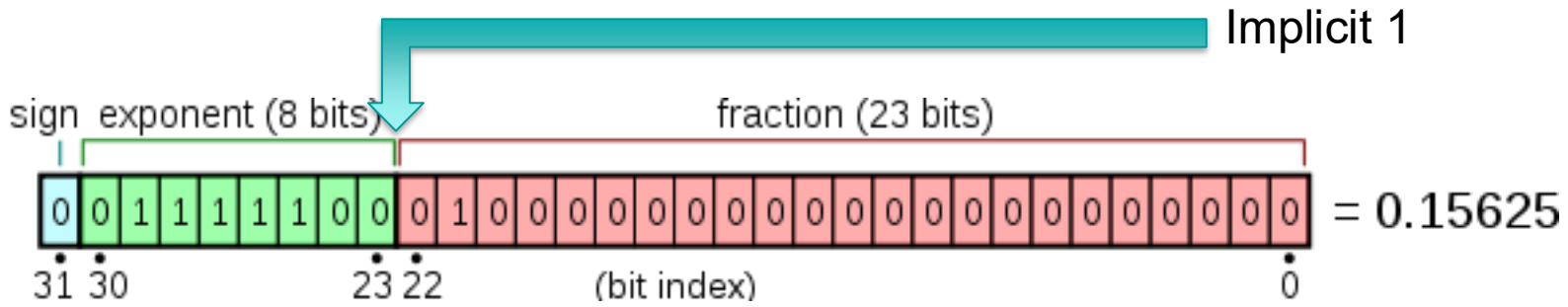
- FP are well suited to perform arithmetic on a computer
  - Since 1985 FP representations and operations are standardized in IEEE754 (https://en.wikipedia.org/wiki/IEEE_754)

The standard defines:

- *arithmetic formats:* sets of binary and decimal floating-point data, which consist of finite numbers (including signed zeros and subnormal numbers), infinities, and special "not a number" values (NaNs)
- *interchange formats:* encodings (bit strings) that may be used to exchange floating-point data in an efficient and compact form
- *rounding rules:* properties to be satisfied when rounding numbers during arithmetic and conversions
- *operations:* arithmetic and other operations (such as trigonometric functions) on arithmetic formats
- *exception handling:* indications of exceptional conditions (such as division by zero, overflow, etc.)

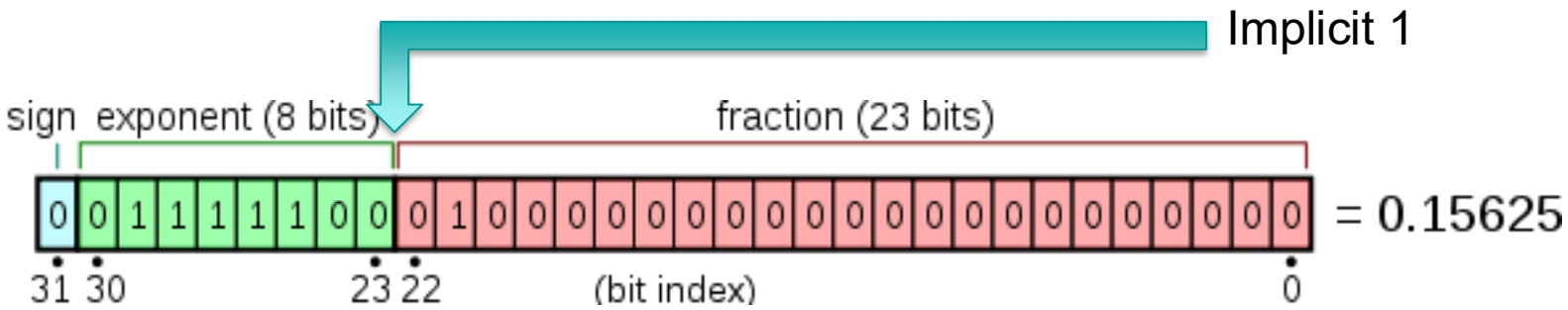
FP binary representations in IEEE754

Format	Bits for the encoding				Exponent bias	Bits precision	Number of decimal digits
	Sign	Exponent	Significand	Total			
Half (binary16)	1	5	10	16	15	11	~3.3
Single (binary32)	1	8	23	32	127	24	~7.2
Double (binary64)	1	11	52	64	1023	53	~15.9
x86 extended	1	15	64	80	16383	64	~19.2
Quadruple (binary128)	1	15	112	128	16383	113	~34.0
Octuple (binary256)	1	19	236	256	262143	237	~71.3



$$n = (-1)^s \times (m2^{-23}) \times 2^{x-127}$$

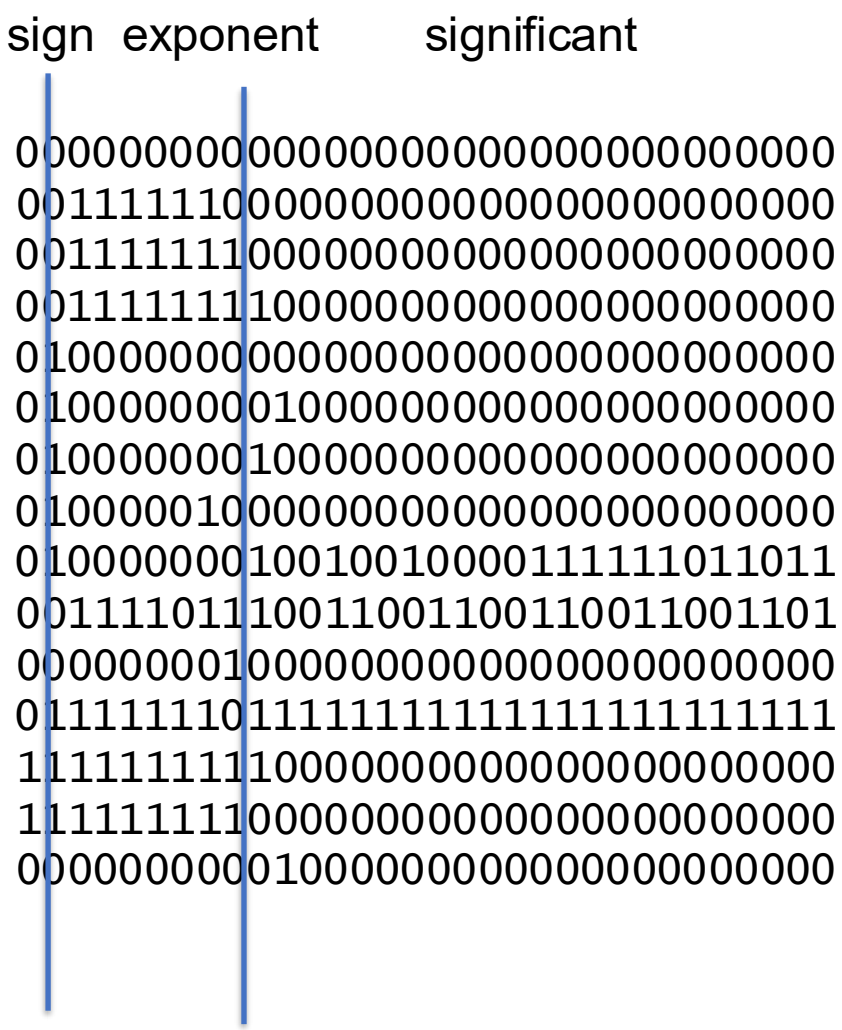
IEEE 754 representation of single precision FP



$$n = (-1)^s \times (m2^{-23}) \times 2^{x-127}$$

Exponent	significand zero	significand non-zero	Equation
00_H	zero, −0	subnormal numbers	$(-1)^{signbits} \times 2^{-126} \times 0.significandbits$
01_H, ..., FE_H	normalized value	normalized value	$(-1^{signbits} \times 2^{exponentbits-127} \times 1.significandbits$
FF_H	±infinity	NaN (quiet,signalling)	

examples



0.00000000e+00	0x0p+0	00000000000000000000000000000000
5.00000000e-01	0x1p-1	00111111000000000000000000000000
1.00000000e+00	0x1p+0	00111111100000000000000000000000
1.50000000e+00	0x1.8p+0	00111111110000000000000000000000
2.00000000e+00	0x1p+1	01000000000000000000000000000000
2.50000000e+00	0x1.4p+1	01000000010000000000000000000000
3.00000000e+00	0x1.8p+1	01000000100000000000000000000000
8.00000000e+00	0x1p+3	01000001000000000000000000000000
3.14159274e+00	0x1.921fb6p+1	01000000100100100000111111011011
1.00000001e-01	0x1.99999ap-4	00111101111001100110011001101101
1.17549435e-38	0x1p-126	00000000100000000000000000000000
3.40282347e+38	0x1.fffffep+127	01111111011111111111111111111111
nan	nan	11111111110000000000000000000000
-inf	-inf	11111111100000000000000000000000
2.93873588e-39	0x1p-128	00000000010000000000000000000000

C++ interlude

```
#include<iostream>
#include<iomanip>
#include<cmath>
#include<limits>
//  std style
template<typename T>
void print(T x) {
 std::cout << std::hexfloat << x <<' ' // exact binary representation
           <<  std::scientific << std::setprecision(8) << x << ' ' // 8 decimal digits
           <<  std::defaultfloat << x << std::endl;    // 6 decimal digits for float
}

// integer representation
#include<cstring>
#include<bitset>
int f2i(float x) { int i; memcpy(&i,&x,sizeof(float)); return i;} // compiler will optimize
//  use bitset to print in binary (bit by bit)
std::cout << std::bitset<32>(f2i(x)) << std::endl; ;   // bit by bit
```

Limits

- Everything you want to know about an arithmetic type is in
  - std::numeric_limits<T>
  - see http://www.cplusplus.com/reference/limits/numeric_limits/

```
#include <iostream>     // std::cout
#include <limits>       // std::numeric_limits
int main () {
  std::cout << std::boolalpha;
  std::cout << "Minimum value: " << std::numeric_limits<float>::min() << '\n';
  std::cout << "Maximum value: " << std::numeric_limits<float>::max() << '\n';
  std::cout << "Is signed: " << std::numeric_limits<float>::is_signed << '\n';
  std::cout << "significand bits: " << std::numeric_limits<float>::digits << '\n';
  std::cout << "precision: " << std::numeric_limits<float>::epsilon() << '\n';
  std::cout << "has infinity: " << std::numeric_limits<float>::has_infinity << '\n';
  return 0;
}
```

Fixed width floating-point types (since C++23)

https://en.cppreference.com/w/cpp/types/floating-point.html

Types Defined in header `<stdfloat>`	**Literal suffix**	**Predefined macro**	**C language type**	**Type properties**			
				bits of storage	bits of precision	bits of exponent	max exponent
float16_t	`f16` or `F16`	`__STDCPP_FLOAT16_T__`	`_Float16`	16	11	5	15
float32_t	`f32` or `F32`	`__STDCPP_FLOAT32_T__`	`_Float32`	32	24	8	127
float64_t	`f64` or `F64`	`__STDCPP_FLOAT64_T__`	`_Float64`	64	53	11	1023
float128_t	`f128` or `F128`	`__STDCPP_FLOAT128_T__`	`_Float128`	128	113	15	16383
bfloat16_t	`bf16` or `BF16`	`__STDCPP_BFLOAT16_T__`	(N/A)	16	8	8	127

Python interlude

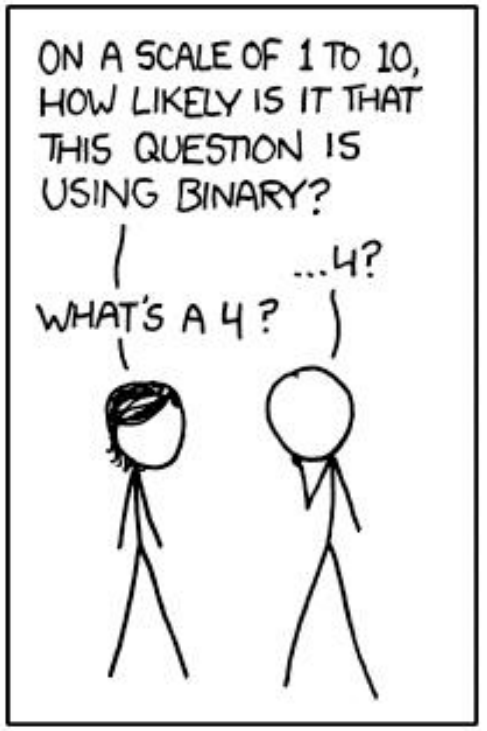
- Python intrinsic “float” is actually double precision
  - float.hex() and float.fromhex() allows to convert to/from exadecimal string
- **numpy** defines float16, float32, float64 (and float128 on x86_64)
  - https://numpy.org/doc/2.3/reference/arrays.scalars.html - floating-point-types
- **decimal** is a module providing FP in base 10 of arbitrary precision
  - https://docs.python.org/3/library/decimal.html

EXERCISES

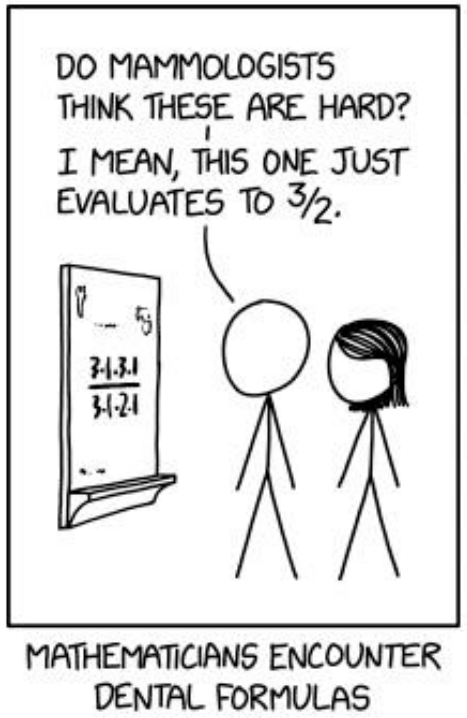
Either in Python or C++

For C++ one can use godbolt (https://godbolt.org/z/d1e18j3jK)

For Python either a python3 shell on your PC, or a jupyter notebook or the browser shell in https://numpy.org/



- Count the number of FP in a binade
  - Verify that it is the same for two different binades
- Compute the ULP for some FPs
  - How it depends on the binade?
- Convert some number (0.1, e, pi, 31.5, …) between various representations (including decimal)
  - See what happens and explain it


DO MAMMOLOGISTS THINK THESE ARE HARD?
I MEAN, THIS ONE JUST EVALUATES TO 3/2.
MATHEMATICIANS ENCOUNTER DENTAL FORMULAS

Arithmetic

FP Arithmetic

- There are two basic operations: sum and multiplication
  - a+b and a×b symbolize the exact sum and multiplication between a and b
  - a⊕b and a⊗b symbolize the rounded sum and multiplication between FPs a and b
    - a+b and a*b are the corresponding operations in code
- We will use decimal precision 2 for all examples to allow mental arithmetic in a finite time
  - 42+3.7 = …
  - 42⊕3.7 = …
  - 42⊕3.3 = … , 42⊕3.5 = … , 43⊕3.5 = …
    42⊕3.3⊕3.4 = … , 3.4⊕3.3⊕42 = …
    42⊕3.3⊕ -45 = …

  (use python decimal to verify results)

- FP Arithmetic is not associative and can lead to loss of precision up to catastrophic cancellations
  - $x + \varepsilon - x \neq \varepsilon$ (can easily be 0 or ∞)

Classical example of loss of precision due to cancellation

For example, consider the quadratic equation:

$$ax^2 + bx + c = 0,$$

with the two exact solutions:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Well know better solution

$$x_1 = \frac{-b - \operatorname{sgn}(b)\sqrt{b^2 - 4ac}}{2a},$$

$$x_2 = \frac{2c}{-b - \operatorname{sgn}(b)\sqrt{b^2 - 4ac}} = \frac{c}{ax_1}.$$

Two sources of cancellation

https://en.wikipedia.org/wiki/Loss_of_significance

A first theorem

- The difference of two number of the same binade is precise
  - 99-11 = 88, 42-37 = 5
  - 0x1.921fb6p+1 - 0x1.001e72p+1 = 0x1.240288p+0

- If 2b>a>b/2>0 then a⊖b = a-b (is precise) (https://en.wikipedia.org/wiki/Sterbenz_lemma)
  - 12-7.4 = 4.6 (precise)
  - 18-7.4= 10.6 => 18⊖7.4 = 11 (not precise)
  - 0x1.333334p-2 ⊖ 0x1.99999ap-3 = 0x1.99999ap-4 (precise)
  - 0x1.333334p-2 ⊖ 0x1.99999ap-4 = 0x1.99999cp-3 (not precise)

Augmented Arithmetic

- If a>b>0 => a+b < 2a

- c = a⊕b
- t = c⊖a is exact
- e = b⊖t is exact
- a+b = c+e
  - e is the error of a⊕b

- This algorithm is known as “fastTwoSum” and (with small modifications) proposed as AugmentedSum in next IEEE754 revision
- A similar algorithm exists for multiplication as well

Example of augmented sum

- $42 \oplus 3.7 = 46$
- $46 \ominus 42 = 4$
- $3.7 \ominus 4 = -0.3$
- *46-0.3 = 42+3.7 (q.e.d.)*
- In short: asum(42,3.7) = 46,-0.3

- $42 \oplus 3.3 \oplus 3.4 = 48$ while 42+3.3+3.4 = 48.7
- asum(42,3.3) = 45,0.3
- asum(45 ,3.4) = 48,0.4
- asum(48,0.3+0.4) = 49, -0.3

Does it help for catastrophic cancellation?

- 42⊕3.3⊕ -45 = 0 while 42+3.3-45 = 0.3
  - Catastrophic cancellation
- asum(42,3.3) = 45,0.3
- asum(45 ,-45) = 0.0,0.0
- asum(0.0,0.3+0.0) = 0.3, 0.0

Compensating algorithms

- Kahan summation
  - https://en.wikipedia.org/wiki/Kahan_summation_algorithm
  - with excruciating details https://arxiv.org/html/2602.19452v1 - LST2

```
T kahanSum(T const * input, size_t n) {
T sum = input[0];
T t = 0.0;
for (size_t i = 1; i<n; ++i)  {
  auto y = input[i] – t;
  auto s = sum + y
  t = (s - sum) – y;
  sum = s;
}
return sum;
```

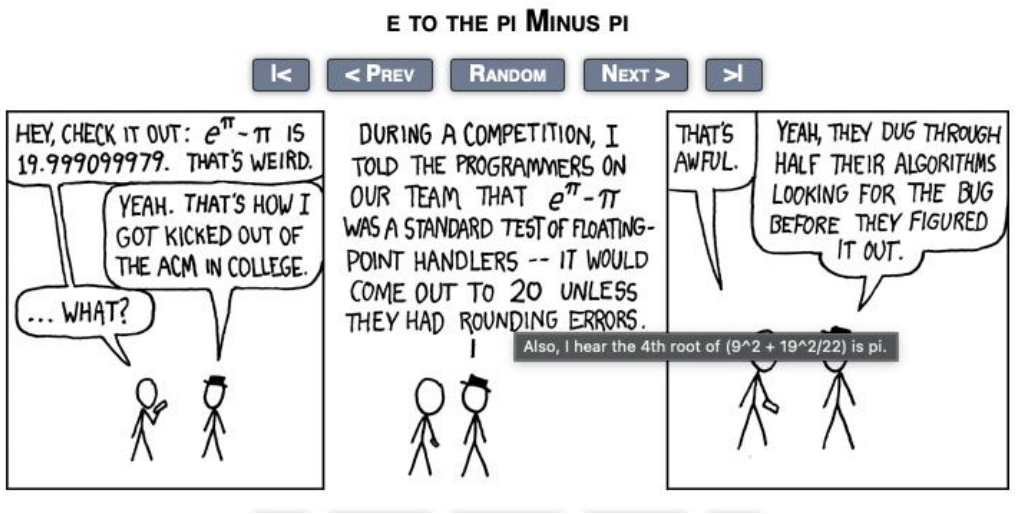
Improved algorithms exist:

- differently handling of various magnitude ranges (“big”, “medium”, “tiny”)
- (partial) sorting
- pairwise recursion
- optimization for parallelism

FMA: Fused Multiple Add

- Also known as MAC (multiply accumulate) instruction on AMD
- fma(a,b,c) computes the correctly rounded result of a*b+c
  - It improves speed and precision in many computation such as linear algebra
  - No standard on how to “contract” expression (depends on compiler)
    - a*b+c*d , a+b*c+d , x*=a;x+=b , etc
  - It clearly introduces asymmetries
    - for instance a*b+c*d ≠ c*d+a*b
- Kahan comes to the rescue again with this algorithm

```
T kahanDet(T a, T b, T c, T d) { // a*d-b*c
  using std::fma;
  auto w = b*c;
  auto e = fma(-b, c, w); // error on w: (w,-e) = augmentedMultiply(b,c)
  auto f = fma(a, d, -w);
  return (f+e);
}
```



EXERCISES

Either in Python or C++

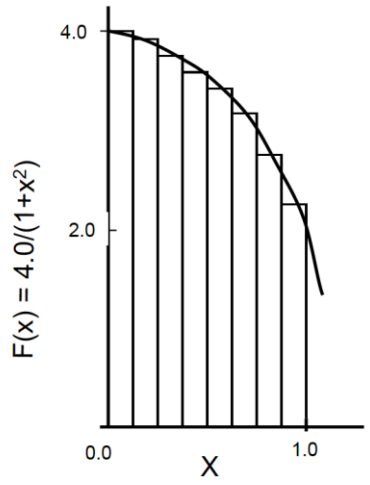
For C++ one can use godbolt (https://godbolt.org/z/d1e18j3jK)

For Python either a python3 shell on your PC, or a jupyter notebook or the browser shell in https://numpy.org/

- Sum T(0.1) 10000 times in binary32, binary64, decimal2, decimal5
  - Google for “patriot failure”
- Sum 10^9 positive random numbers (in range 10^{-n}, 10^{+n})
  - Typical in unbinned log-likehood
- Compute the average age of all humans on earth

- Compute π as the integral of $4/(1+x^2)$ between 0 and 1 (use *float, double, simd, int, mixed*)
  - (credit: Tim Mattson)

Numerical integration



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)}\,dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^{N} F(x_i)\Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i.

```
static long num_steps = 100000;
double step;
int main ()
{       int i;      double x, pi, sum = 0.0;

        step = 1.0/(double) num_steps;

        for (i=0;i< num_steps; i++){
                x = (i+0.5)*step;
                sum = sum + 4.0/(1.0+x*x);
        }
        pi = step * sum;
}
```

- Solve the equation $x^2 - 2bx + 1 = 0$ using the "high-school" formula
  - What is the smallest value of b (single precision) for which one of the solution becomes 0?
  - What is instead the correct result? (use only single precision computations)

Random Number interlude

```
int getRandomNumber()
{
    return 4;  // chosen by fair dice roll.
               // guaranteed to be random.
}
```

```
#include <iostream>
#include <iomanip>
#include <random>
#include <algorithm>
#include <cmath>
int main()
{
    std::random_device rd;  // to obtain a seed for the random number engine
    std::mt19937 gen(rd()); // Standard mersenne_twister_engine seeded with rd()
    std::uniform_real_distribution<float> dis(-1.0, 1.0);
    size_t N = 40*1000*1000;
    float * x = new float[N];
    for (int n = 0; n < N; ++n) {
        // Use dis to transform the random unsigned int generated by gen into a
        // float in [-1, 1). Each call to dis(gen) generates a new random float.
        x[n] = dis(gen);
    }
    // sort in absolute value
    std::sort(x,x+N,[](float a, float b) { return std::abs(a) < std::abs(b);});
    // print first and last 40
    for (int i=0; i<40; ++i) std::cout << x[i] << ' '; std::cout << '\n';
    for (int i=0; i<40; ++i) std::cout << x[N-1-i] << ' '; std::cout << '\n'; std::cout << '\n';
    std::cout << std::hexfloat;
    for (int i=0; i<40; ++i) std::cout << x[i] << ' '; std::cout << '\n';
    for (int i=0; i<40; ++i) std::cout << x[N-1-i] << ' '; std::cout << '\n';

    delete [] x;
    return 0;
}
```

```
0 -5.96046e-08 1.19209e-07 -1.19209e-07 1.19209e-07 -1.19209e-07 1.19209e-07 1.19209e-07 -1.78814e-07 2.38419e-07 -2.38419e-07 -3.57628e-07 3.57628e-07 3.57628e-07 -4.17233e-07 -4.17233e-07 -4.17233e-07 4.76837e-07 4.76837e-07 4.76837e-07 5.96046e-07 -5.96046e-07 -6.55651e-07 7.15256e-07 -7.15256e-07 -7.7486e-07 8.34465e-07 8.34465e-07 -8.34465e-07 8.34465e-07 -8.9407e-07 -9.53674e-07 -9.53674e-07 -9.53674e-07 9.53674e-07 1.07288e-06 1.07288e-06 1.19209e-06 1.19209e-06 1.19209e-06
1 -1 1 1 1 -1 1 1 1 1 -1 -1 -1 -1 -1 1 1 0.999999 0.999999 -0.999999 -0.999999 -0.999999 -0.999999 -0.999999 0.999999 0.999999 0.999999 0.999999 -0.999999 -0.999999 -0.999999 0.999999 0.999999 0.999999 -0.999999 -0.999999 -0.999999 0.999999 0.999999 0.999999 -0.999999

0x0p+0 -0x1p-24 0x1p-23 -0x1p-23 0x1p-23 -0x1p-23 0x1p-23 0x1p-23 -0x1.8p-23 0x1p-22 -0x1p-22 -0x1.8p-22 0x1.8p-22 0x1.8p-22 -0x1.cp-22 -0x1.cp-22 -0x1.cp-22 0x1p-21 0x1p-21 0x1p-21 0x1.4p-21 -0x1.4p-21 -0x1.6p-21 0x1.8p-21 -0x1.8p-21 -0x1.ap-21 0x1.cp-21 0x1.cp-21 -0x1.cp-21 0x1.cp-21 -0x1.ep-21 -0x1p-20 -0x1p-20 -0x1p-20 0x1p-20 0x1.2p-20 0x1.2p-20 0x1.4p-20 0x1.4p-20 0x1.4p-20
0x1p+0 -0x1p+0 0x1p+0 0x1p+0 0x1p+0 -0x1.fffffep-1 0x1.fffffcp-1 0x1.fffffcp-1 0x1.fffff8p-1 0x1.fffff4p-1 -0x1.fffff4p-1 -0x1.fffff2p-1 -0x1.fffff2p-1 -0x1.fffffp-1 -0x1.fffffp-1 0x1.fffffp-1 0x1.fffffp-1 0x1.ffffecp-1 0x1.ffffecp-1 -0x1.ffffeap-1 -0x1.ffffeap-1 -0x1.ffffeap-1 -0x1.ffffeap-1 -0x1.ffffeap-1 0x1.ffffe8p-1 0x1.ffffe8p-1 0x1.ffffe8p-1 0x1.ffffe8p-1 -0x1.ffffe6p-1 -0x1.ffffe6p-1 -0x1.ffffe6p-1 0x1.ffffe4p-1 0x1.ffffe4p-1 -0x1.ffffe2p-1 -0x1.ffffe2p-1 -0x1.ffffep-1 0x1.ffffep-1 0x1.ffffep-1 0x1.ffffep-1 -0x1.ffffep-1
```

Uniform vs canonical

(for a detailed discussion see among others https://github.com/DiscreteLogarithm/canonical-random-float/tree/master)

Uniform in range [a,b)

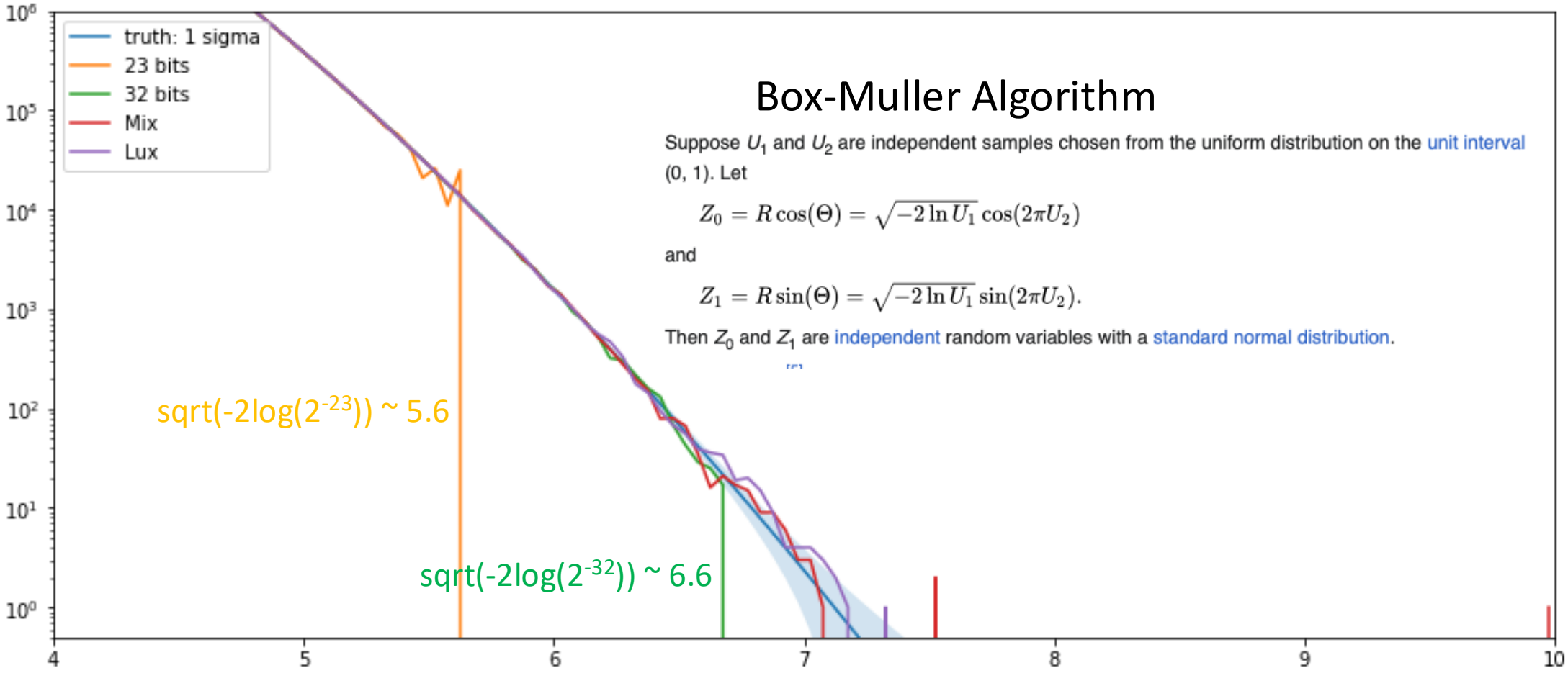
- Throw 32 coins: interpret as a positive integer N
- Compute r = float(N)/float(2^{32})*(b-a) –a

Canonical

- Throw 23 coins: set significand of r to them
- Throw one more coin: if 1 set exponent of r to -1: *uniform in [0.5,1) prob. 1/2*
  - If 0 :Throw one more coin: if 1 set exponent of r to -2: *uniform in [0.25,0.5) prob. 1/4*
    - If 0: Throw one more coin: if 1 set exponent of r to -3: *uniform in [0.125,0.25) prob. 1/8*
      - If 0: Throw one more coin: if 1 set exponent of r to -4: *uniform in [0.0625,0.125) prob. 1/16*
        - If 0: ….
- If symmetric interval around 0 needed : use last coin to choose sign

Gaussian Tail

- Generated $256*10^9$ Normal distributed using Box-Muller
- Mix: 23 bits for U2, 41 bits for U1
- Lux: all-possible floats for both



Elementary Functions

Elementary functions

- IEEE754 defines a set of “additional operations” (aka elementary functions)
  - exp, log, power of various forms
  - trig, hyperbolic and their inverse (arc)
  - They are “required” to be correctly rounded
- C and C++ std define some more special functions
  - https://en.cppreference.com/cpp/numeric/math
  - https://en.wikipedia.org/wiki/C_mathematical_functions
- Each vendor/compiler provides its own implementation
  - A comparison of their accuracy is compiled by B.Gladman et al.
    - https://members.loria.fr/PZimmermann/papers/accuracy.pdf
- The most accurate implementation to date is from the “core-math” project
  - https://core-math.gitlabpages.inria.fr/

Library functions

library	GNU libc	IML	AMD	Newlib	OpenLibm	Musl	Apple	LLVM	CUDA	ROCm
version	2.35	2021.2.0	3.8	4.2.0	0.8.1	1.2.2	12.1	13.0.0	11.5.0	4.5.2
acos	0.899	**0.528**	0.897	0.899	0.918	0.918	0.634	NA	1.69	1.47
acosh	2.01	**0.501**	0.504	2.01	2.01	NaN	0.502	NA	2.18	0.564
asin	0.898	**0.528**	0.781	0.926	0.743	0.743	0.634	NA	2.05	2.54
asinh	1.78	0.527	**0.518**	1.78	1.78	1.78	0.515	NA	1.78	0.573
atan	0.853	0.541	**0.501**	0.853	0.853	0.853	0.722	NA	2.06	2.10
atanh	1.73	0.507	**0.506**	1.73	1.73	1.73	0.511	NA	3.16	0.574
cbrt	0.969	0.520	0.548	3.56	**0.500**	**0.500**	**0.500**	NA	1.17	1.14
cos	0.561	0.548	0.729	2.91	**0.501**	**0.501**	0.862	0.776	1.52	1.61
cosh	1.89	**0.506**	8.39e6	2.51	1.36	1.03	0.589	NA	2.34	0.567
erf	0.968	0.507	0.968	0.968	0.943	0.968	**0.501**	NA	1.14	2.48
erfc	3.13	**0.502**	3.13	63.9	3.17	3.13	0.750	NA	4.49	3.33
exp	0.502	0.506	**0.501**	1.94e4	0.911	0.502	0.576	0.502	1.94	1.00
exp10	**0.502**	0.507	1.00	1.06	NA	3.88	0.580	NA	2.07	1.00
exp2	0.502	0.519	**0.501**	1.02	**0.501**	0.502	0.570	0.502	2.39	0.871
expm1	0.813	0.544	**0.536**	0.813	0.813	0.813	0.687	1.50	1.45	1.45

Maximal error (in bits) for single precision functions as implemented in various math library

- https://members.loria.fr/PZimmermann/papers/accuracy.pdf

Evaluating a function

for a recent review see https://dl.acm.org/doi/pdf/10.1145/3747840

Five steps

1. Handling special values, under/over-flows and invalid
   - Standard specify them all!
2. Range Reduction
3. Approximation
4. If required:
   - precision test and handling of “Table Maker’s Dilemma”
5. Back to full range

Range reduction

- exp, log, pow
  - exploit fp representation: $x = 1.m \cdot 2^n$
  - For log(x): $x = y \cdot 2^z$, with y in [0.75,1.5) . $\log(x) = z \cdot \log(2) + \log(y)$
  - For exp(x): $x = k \cdot \log(2) + y$, with y in [-log(2)/2;log(2)/2). $\exp(x) = 2^k \cdot \exp(y)$
  - $x^y = e^{y \cdot \log(x)}$
- sin,cos,tan,atan(2)
  - Reduce to [0, π/4) (trivial for f(π·x) known as sinpi,cospi etc, tricky for sin(x) etc)
  - use trig identities to go back
- asin,acos, hyperbolic
  - approximate it in “ad-hoc” intervals (make sure continuity at edges)

Approximation

- Since 1964 the reference for function approximation is the *Abramowitz and Stegun*
  - Today replaced by the *digital library* https://dlmf.nist.gov/
- Elementary functions are usually approximated either by
  - shift-and-add algorithms (CORDIC https://en.wikipedia.org/wiki/CORDIC)
    - Today limited to FPGA
  - a polynomial
    - Chebyshev approximation satisfying the ”superior” norm (max in an interval)
  - a rational function (ratio of two polynomials)
    - Pade’ approximation
  - an iterative method such as Newton-Raphson

See also the short paper by J.M.Muller https://hal.science/hal-02517784v1/file/approxfunctions.pdf

1/x, √(x),1/√(x) aka rcp(x),sqrt(x),rsqrt(x)

- Few fast converging Newton-Raphson iterations
  - Starting point is an approximation of $f(1.m)$

The major contribution of game industry to CS is the discovery of the “magic” fast 1/√x algorithm in quake3

```
float rsqrt(float x){
  union {float f;int i;} tmp;
  tmp.f = x;
  tmp.i = 0x5f3759df - (tmp.i >> 1);
  float y = tmp.f; // approximate
  return y * (1.5f - 0.5f * x * y * y); // NR
}
```

Real x86_64 code for 1/√x

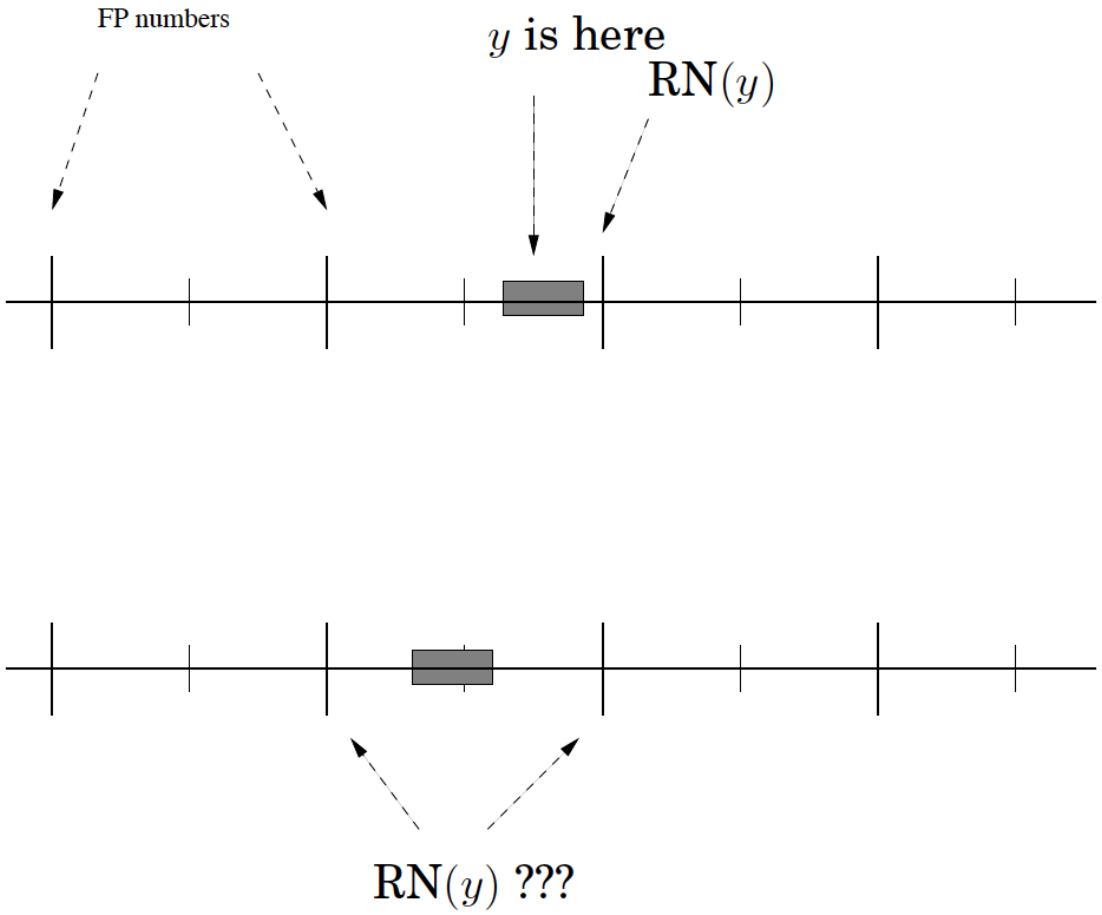
```
_mm_store_ss( &y, _mm_rsqrt_ss( _mm_load_ss( &x ) ) );
return y * (1.5f - 0.5f * x * y * y); // One round of Newton's method
```

Real x86_64 code for 1/x

```
_mm_store_ss( &y,_mm_rcp_ss( _mm_load_ss( &x ) ) );
 return (y+y) – x*y*y; // One round of Newton's method
```

https://en.wikipedia.org/wiki/Fast_inverse_square_root

Table Maker's Dilemma



to guarantee a given precision one needs to evaluate the error of the approximation "y = f(x)"

- From first principle
- Or on the fly as seen before

1. The function need to be computed with at least one more digit (the round digit)
2. In Round2Near: if the error interval includes the middle point an even more precise evaluation is required
   - the round digit may be followed by a sequel of 0's or 1's (or 9's) and the TMD solution may be very expensive

Examples :

sin(0.40225342) = 0x1.90e382fffff7bp-2 sin(0.47485355) = 0x1.d42e63000000bp-2

EXERCISES

Either in Python or C++

For C++ one can use godbolt (https://godbolt.org/z/d1e18j3jK)

For Python either a python3 shell on your PC, or a jupyter notebook or the browser shell in https://numpy.org/

Polynomials

- Approximate sin(x) in [0, π/4) with polynomials of increasing order
  - Taylor
  - Chebyshev
  - Pade
- Plot the absolute and relative error w/r/t a high precision result

- Use python mpmath module https://mpmath.readthedocs.io/en/latest/calculus/approximation.html
- or sage, or mathematica…

- Test rsqrt precision in the interval [0.5, 4)
  - use “quake magic” and/or x86_64 intrinsics plus multiple NR iterations
  - Try to reach faithful (or even correct) rounding.

- Compute sin(0x1.e64002p-2) using Taylor expansion:
  - How many terms are needed to reach faithful rounding in single precision?
  - How many more terms to reach correct rounding (in single precision)?
  - Use double precision to compute, then round to single

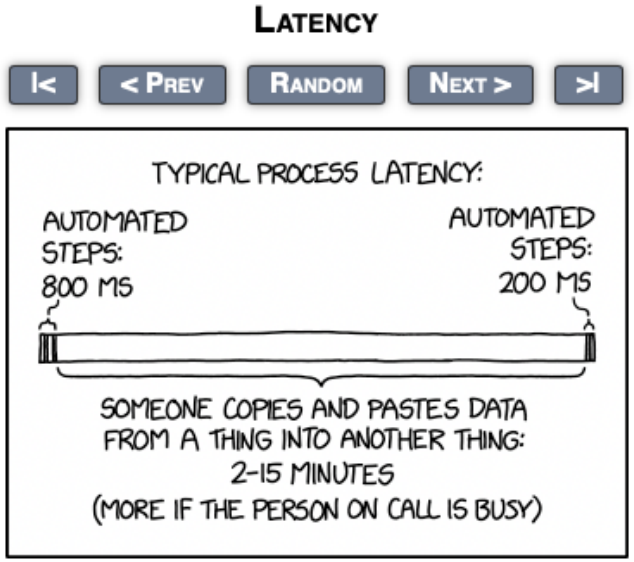
TMD in decimal

n	x	$\cos(x)$
4	1.149	$0.409400000428\ldots$
5	1.6406	$-0.697470000000219\ldots$
6	1.41162	$0.15850500000180\ldots$
7	1.225910	$0.338089700000005907\ldots$
8	1.9509449	$-0.37105844000000012\ldots$

Table 1: Worst cases for n-digit mantissa, radix-10, floating-point numbers between 1 and 2, directed rounding modes and the cosine function.

Use cos(x) approximation in decimal (taylor)
see https://docs.python.org/3/library/decimal.html - recipes

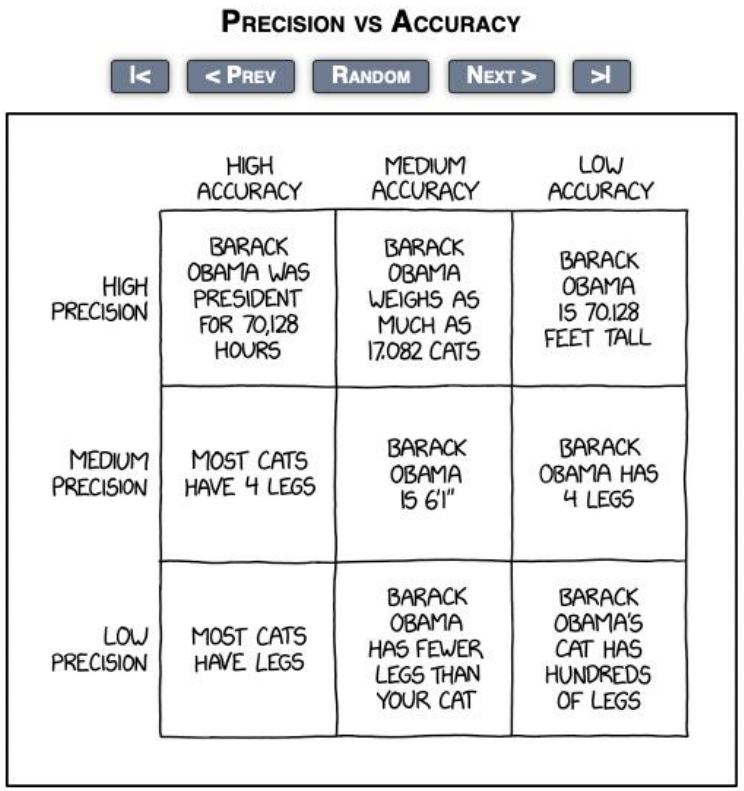
to “solve” TMD above (mind: direct rounding!)



Precision, Accuracy, Speed

Definitions

- Precision
  - Numerical precision (in bits)
    - For an instrument (ruler!) typically “half a tick”
    - Comes from specs
- Accuracy
  - The error w/r/t the truth
  - Can be evaluated comparing with a “refence” or a different algorithm
    - In science “comparing different experiments”
- Target Accuracy
  - The acceptable tolerance w/r/t the above
  - Evaluated, for instance, w/r/t know error on the truth



Cost of operations (in cpu cycles on x86_64)

https://www.agner.org/optimize/instruction_tables.pdf (icelake & zen5)

		latency		inv throughput	
op	instruction	s	d	s	d
+,-	ADD,SUB	3-4	3-4	0.5	0.5
==, <, >	COMISS CMP..	2,3	2,3	0.5-1	0.5-1
f=d, d=f ,i=f, f=i	CVT..	3-7	3-7	0.5-1	0.5-1
|,&,^	AND, OR …	1-2	1-2	0.25-0.5	0.25-0.5
*	MUL	5-4	5-4	0.5	0.5
a*b+c	FMA	4	4	0.5	0.5
/,sqrt	DIV, SQRT	11-20	13-29	3-12	4-24
1.f/ , 1.f/sqrt	RCP, RSQRT	4-5		0.5-1	
=	MOV	1,3,…	1,3,…	0.25,1,….	0.25,1,…

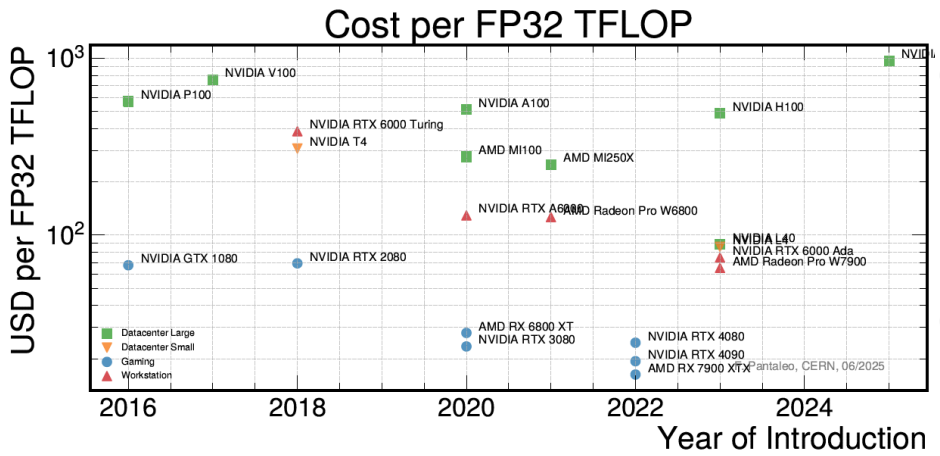
+ jump "overhead" (mispredicted branches)

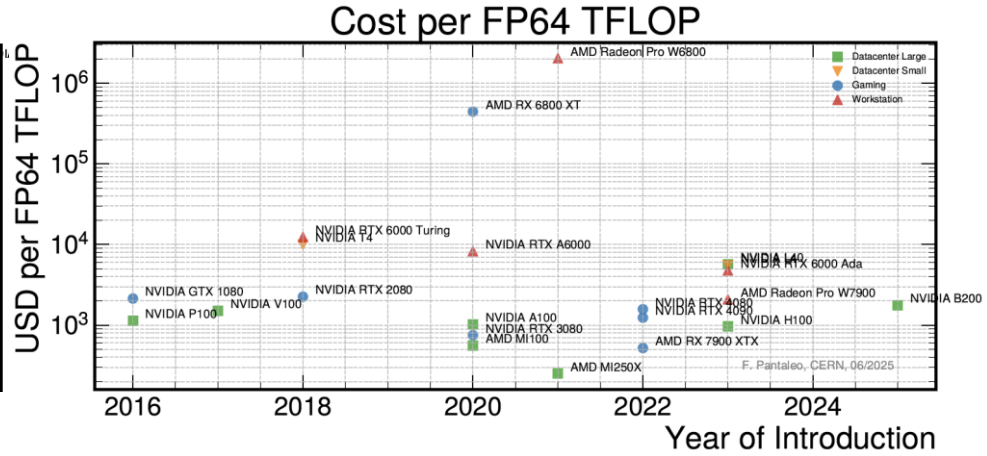
(does not vectorize on intel?)

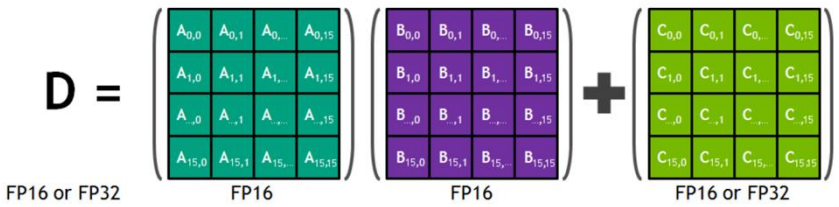
➔350 from main memory

GPU landscape:

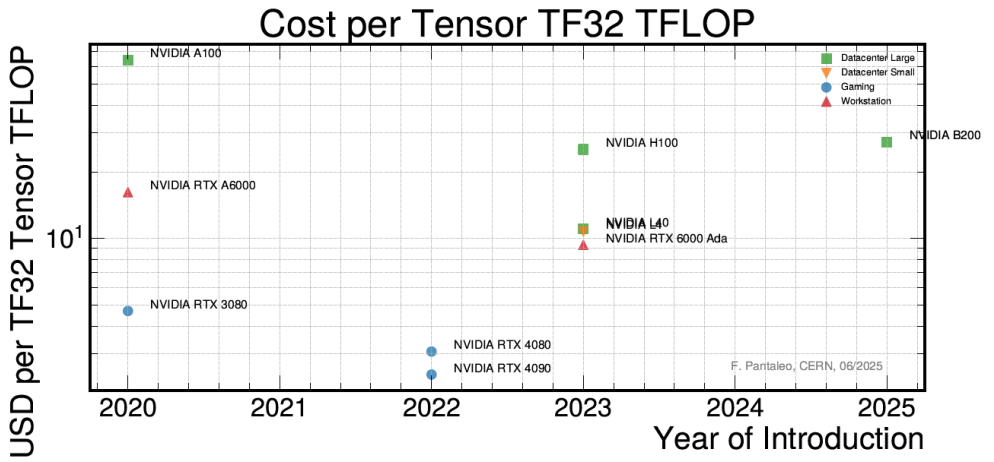
FP64 is 32 times slower than FP32 on affordable GPU







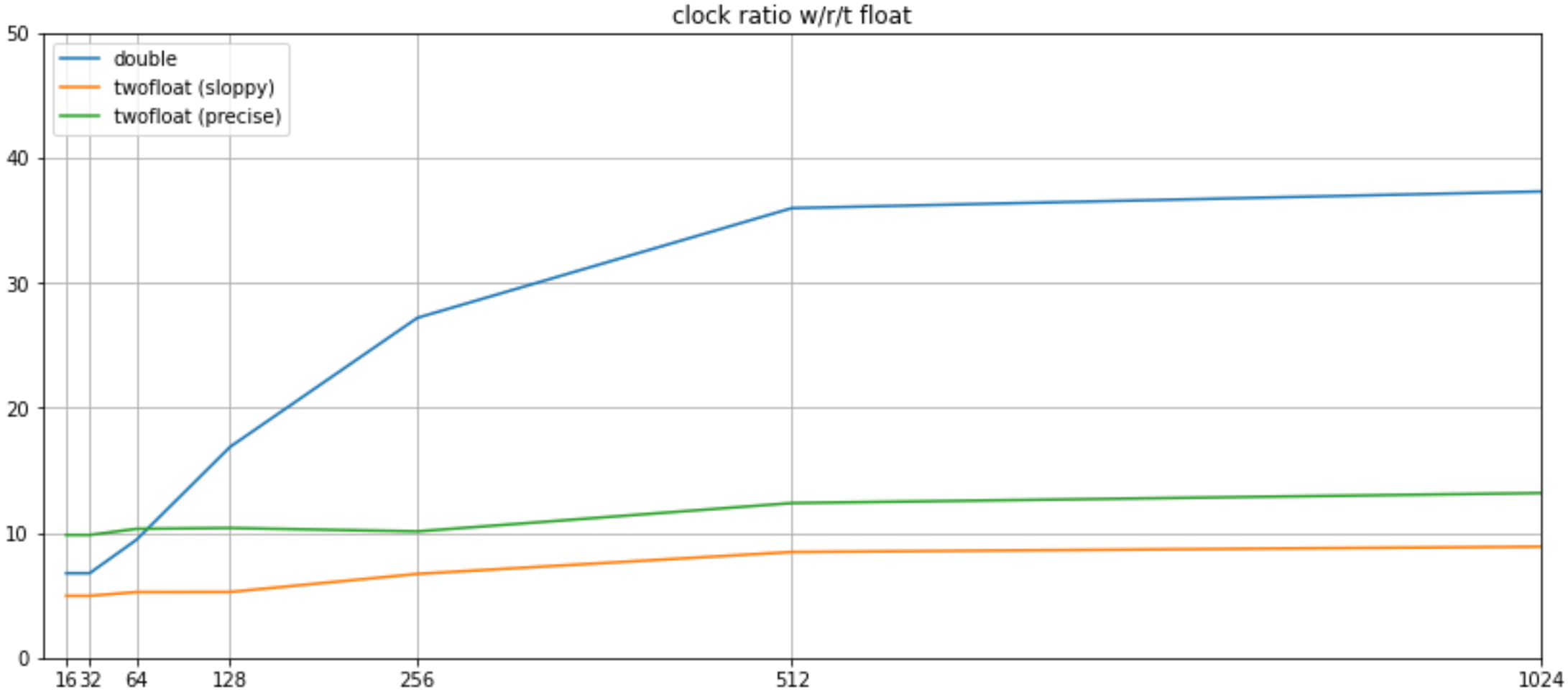
min rank for tensor is 16 (not 4)!



GPU (T4) performance

Brute-force inversion of 5x5 Pos-Def Matrix using Cholesky decomposition

33 + , 29 - , 126 * , 5 / , no sqrt (compiler optimization and fma will contract some)

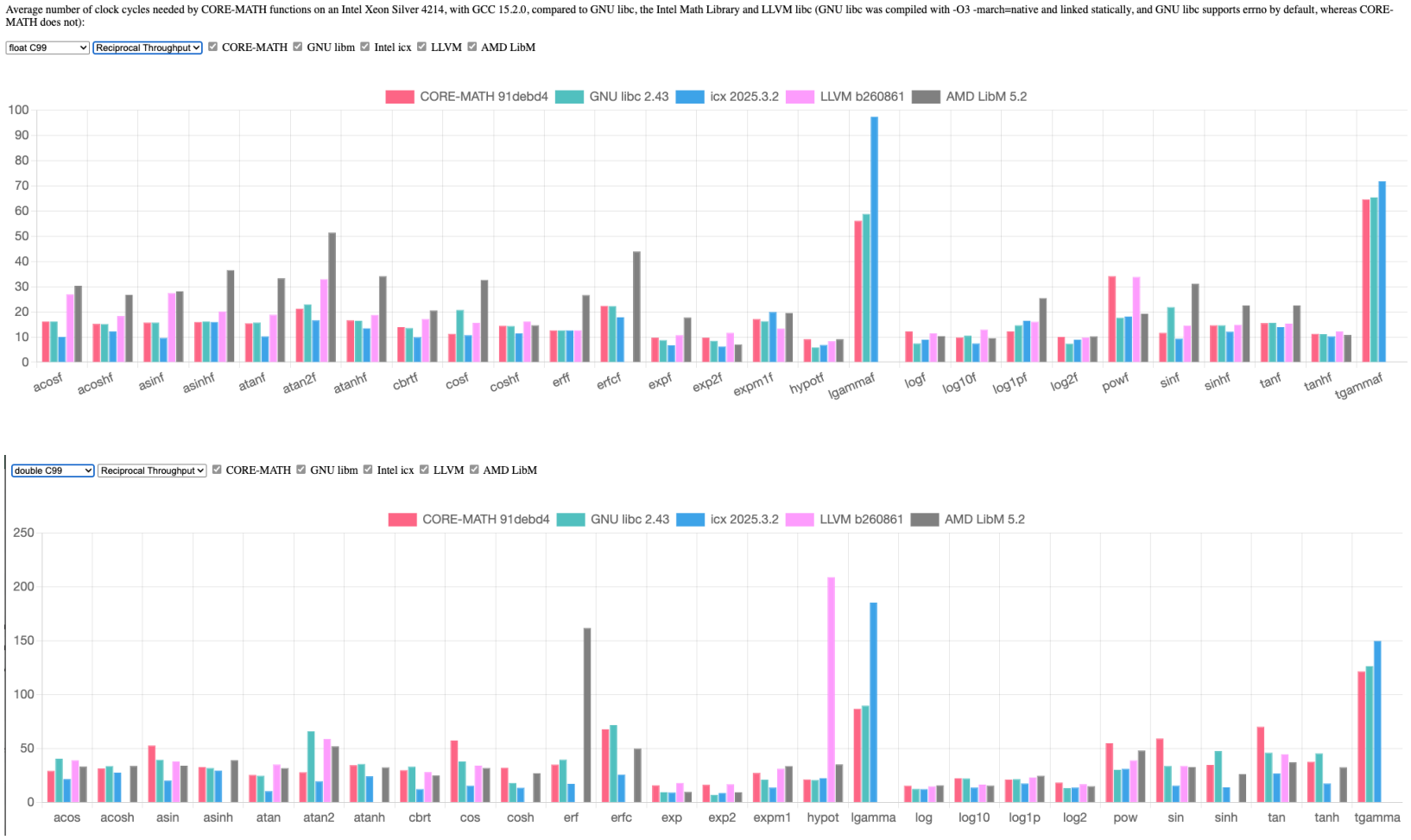


float vs double: summary

- In the old days: X87
  - Double faster than float!
- X86_64 (and ARM, PPC, RISC)
  - Scalar: float and double same speed (but div, sqrt, memory bandwidth)
  - Vector: float twice faster than double
- GPU: gaming and “Data Center”
  - Float 64 times faster than double
- GPU: HPC
  - Float twice faster than double
- GPU: AI
  - Optimized for low-precision tensor operations (GEMM can profit)

Elementary functions 1/throughput (in cycles)

(origin https://core-math.gitlabpages.inria.fr/)



Precision vs Speed for elementary functions

- Log
  - 24 bit precision needs 8th degree polynomial
  - 16 bit precision needs 5th degree polynomial
  - 7 bit precision needs 2th degree polynomial

	2	3	4	5	6	7	8
e^x	6.8	10.8	15.1	19.8	24.6	29.6	34.7
$\sin(x)$	7.8	12.7	16.1	21.6	25.5	31.3	35.7
$\ln(1+x)$	8.2	11.1	14.0	16.8	19.6	22.3	25.0
$\arctan(x)$	8.7	9.8	13.2	15.5	17.2	21.2	22.3
$\tan(x)$	4.8	6.9	8.9	10.9	12.9	14.9	16.9
$\arcsin(x)$	3.4	4.0	4.4	4.7	4.9	5.1	5.3
$\sqrt{x}$	3.9	4.4	4.8	5.2	5.4	5.6	5.8

Number of significant bits of accuracy vs degree of minimax polynomial approximating the range [0..1].

Example: multiple scattering formula

```
double ms(double radLen, double m2, double p2) {
  constexpr double amscon = 1.8496e-4;    // (13.6MeV)**2
  double e2     = p2 + m2;
  double beta2  = p2/e2;
  double fact = 1 + 0.038*log(radLen);  fact *=fact;
  double a = fact/(beta2*p2);
  return amscon*radLen*a;
}
```

Already an approximation

Material density, thickness, track angle
Known at percent?

```
float msf(float radLen, float m2, float p2) {
  constexpr float amscon = 1.8496e-4;    // (13.6MeV)**2
  float e2     = p2 + m2;

  float fact = 1.f + 0.038f*unsafe_logf<2>(radLen); fact /= p2;
  fact *=fact;
  float a = e2*fact;
  return amscon*radLen*a;
}
```

2nd order polynomial

Cash-Karp Runge-Kutta Step

3. A STRATEGY FOR DEALING WITH NONSMOOTH BEHAVIOR

The Runge-Kutta formula derived in the previous section has the special property that it contains imbedded solutions of all orders less than five. In addition, the formula has been designed so that the first five c_i values span the range [0, 1] with reasonable uniformity, so that we have a very good chance of spotting bad behavior in f if it occurs. Our aim is to derive an automatic strategy that allows us to quit early, i.e., before all six function evaluations have been computed on the current step, if we suspect trouble, and to accept a lower order solution if appropriate.

We assume that we have computed a numerical solution y_{n-1} at the step point x_{n-1} and that for the current step, from x_{n-1} to $x_n = x_{n-1} + h$, all six function evaluations are computed so that solutions of all orders from 1 to 5 are available. (We guarantee this situation for the first step with $n = 1$). We denote the imbedded solution of order i at x_n by $y_n^{(i)}$, $1 \le i \le 5$, and define

$$\mathrm{ERR}(n, i) = \| y_n^{(i+1)} - y_n^{(i)} \|^{1/(i+1)}, \qquad \text{for} \quad i \in 1, 2, 4. \tag{6}$$

We exclude the case $i = 3$ for two reasons. First, following the approach of Shampine et al. [15], we allow only a few different orders to be used, and we have chosen to allow orders 2, 3, or 5. Second, $\mathrm{ERR}(n, 3)$ is of no use in predicting when to quit early since all six k_i's are required before $y_n^{(4)}$ can be computed.

Suppose now that we were to accept the solution of order 5 at x_n. We wish to compute a suitable step length, $\bar{h}_4$, to be used in integrating from x_n to x_{n+1} using a 5(4) formula. A typical step-choosing strategy would compute $\bar{h}_4$ as

$$\bar{h}_4 = \frac{\mathrm{SF} \times h}{\mathrm{E}(n, 4)}, \qquad \text{where} \quad \mathrm{E}(n, 4) = \frac{\mathrm{ERR}(n, 4)}{\epsilon^{1/5}}. \tag{7}$$

Here ϵ is the local accuracy required (as specified by the user) and SF is a safety factor often taken to be 0.9. Similarly, if we were to accept either the second- or third-order solution at x_n, the steplengths $\bar{h}_1$, $\bar{h}_2$, respectively, that would be selected at the next step by our step-control algorithm would be

$$\bar{h}_i = \frac{\mathrm{SF} \times h}{\mathrm{E}(n, i)}, \qquad \text{where} \quad \mathrm{E}(n, i) = \frac{\mathrm{ERR}(n, i)}{\epsilon^{1/(i+1)}}, \quad i \in 1, 2. \tag{8}$$

0.9*step/pow(err/eps,0.2)

Code pitfalls
and opportunity for optimization

Language and compiler “rules”

- In essentially any language default FP arith is in double
  - Any other precision needs to be explicitly stated
  - One element in double and the whole expression will be evaluated in double
    - Promotion to double can be turn in a warning using “*-Wdouble-promotion”*
- C and C++ compilers (gcc, clang) by default will never reorder or transform expressions if the result may change
  - FMA is (currently) an exception
- C is a language born in the late `70
  - even C++ does not (potentially) break old code

Compiler option affecting FP Arith

https://gcc.gnu.org/wiki/FloatingPointMath
https://clang.llvm.org/docs/UsersManual.html - controlling-floating-point-behavior

- -fno-errno-math
  - Assume user ignore libc error flag
- -fno-trapping-math
  - Assume user ignore IEEE754 exceptions
- -fno-signed-zeros
  - Assume user do not distinguish sign of zeroes
- -finite-math-only
  - Assume code does not emit NaNs and Infs
- -fassociative-math
  - Allows free reordering of expressions
- -freciprocal-math
  - Allows transformation of divisions
- **-ffast-math**
  - All above and more

Controlling fma (contraction)

- gcc (and clang) flags
  - `-ffp-contract=off` (default is "`fast`")
  - https://stackoverflow.com/questions/43352510/difference-in-gcc-ffp-contract-options
- cuda (nvcc) flags
  - `-fmad=false` (default is "`true`")
  - https://docs.nvidia.com/cuda/floating-point/index.html
- fma function itself will in any case honored and will use the hardware instruction if supported

Inspecting generated code

objdump -S -r -C --no-show-raw-insn –w kernel.o | less
(on MacOS: otool -t -v -V –X kernel.o | c++filt | less)
Or use http://gcc.godbolt.org

c++ -S kernel.cc; less kernel.s

```
float
kernel(float a, float x, float y)
{
  return a*x + a*y;
}
```

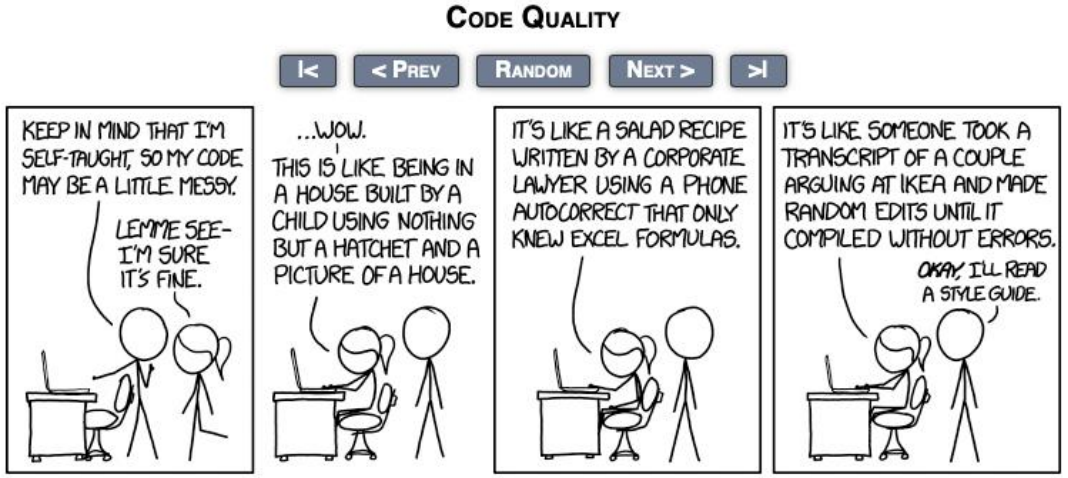
-O2

```
kernel(float, float, float):
        mulss   %xmm0,%xmm2
        mulss   %xmm0,%xmm1
        addss   %xmm2,%xmm1
        movaps  %xmm1,%xmm0
        ret
```

-O2,-ffast-math

```
kernel(float, float, float):
        addss   %xmm2,%xmm1
        mulss   %xmm0,%xmm1
        movaps  %xmm1,%xmm0
        ret
```

Formula Translation: what was the author's intention?




```
// Energy loss and variance according to Bethe and Heitler, see also
// Comp. Phys. Comm. 79 (1994) 157.
//
double p = mom.mag();  //  sqrt(mom.x()*mom.x()+mom.y()*mom.y()+mom.z()*mom.z())
double normalisedPath = fabs(p/mom.z())*radLen;
double z = exp(-normalisedPath);
double varz = (exp(-normalisedPath*log(3.)/log(2.))- exp(-2*normalisedPath));
```

```
double pt = mom.perp();    //  sqrt(mom.x()*mom.x()+mom.y()*mom.y())
double j = a_i*(d_x * mom.x() + d_y * mom.y())/(pt*pt);
double r_x = d_x - 2* mom.x()*(d_x*mom.x()+d_y*mom.y())/(pt*pt);
double r_y = d_y - 2* mom.y()*(d_x*mom.x()+d_y*mom.y())/(pt*pt);
double s = 1/(pt*pt*sqrt(1 - j*j));
```

EXERCISES

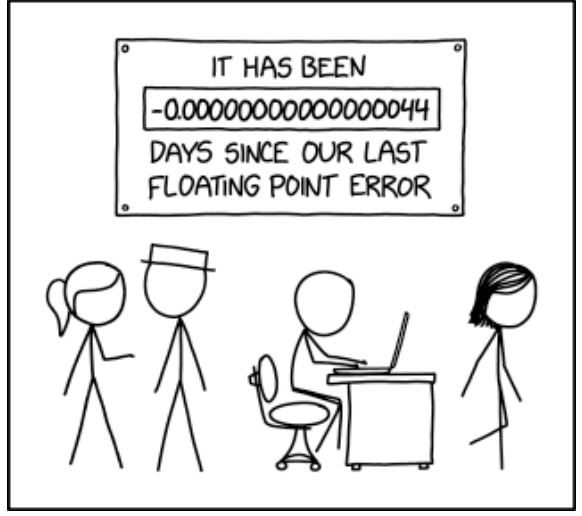
In C++

use godbolt (https://godbolt.org/z/5TWjsKqha)

change compiler options for various compilers and see what happens

Try to optimize some code and see if it really improves

Anecdotes



- Floating point “disasters”
  - http://ta.twi.tudelft.nl/users/vuik/wi211/disasters.html
  - https://www.iro.umontreal.ca/~mignotte/IFT2425/Disasters.html
  - https://slate.com/technology/2019/10/round-floor-software-errors-stock-market-battlefield.html
  - https://medium.com/@sohail_saifii/the-floating-point-standard-thats-silently-breaking-financial-software-7f7e93430dbb
  - https://pavpanchekha.com/blog/casio-mathjs.html

References

- What Every Computer Scientist Should Know About Floating-Point Arithmetic (1991) https://dl.acm.org/doi/pdf/10.1145/103162.103163
- Handbook of Floating-Point Arithmetic(2018) https://link.springer.com/book/10.1007/978-3-319-76526-6
- Elementary Functions (2016) https://link.springer.com/book/10.1007/978-1-4899-7983-4
- Floating-point arithmetic (2023) https://www.cambridge.org/core/journals/acta-numerica/article/floatingpoint-arithmetic/287C4D5F6D4A43FBEEB1ABED2A405AAF
- Low Precision (2025) https://www.siam.org/publications/siam-news/articles/mixed-precision-computing-high-accuracy-with-low-precision/
- Bleeding edge
  - IEEE Arith Conference: https://arith2026.org/archive.html
  - FPTalks: https://fptalks.org/